

MEDIATEK

MT8668 Hypervisor OP-TEE User Manual

版本: 1.0
出版日期: 2026-01-28

Use of this document and any information contained therein is subject to the terms and conditions set forth in Exhibit 1. This document is subject to change without notice.

版本记录

版本	日期	作者	描述
1.0	2026-01-28	陈琰	正式版

目录

版本记录	2
目录	3
图片目录	4
表格目录	4
1 概述	5
1.1 TrustZone 功能简介	5
1.2 OP-TEE 简介	7
2 OP-TEE 框架和功能	9
2.1 开启虚拟化后 OP-TEE 核心组件	9
2.2 MediaTek 平台 OP-TEE 开启方式	12
2.2.1 Yocto Branch	12
2.2.2 Android Branch	13
2.3 安全存储机制	13
2.3.1 REE 文件系统	14
2.3.2 RPMB 文件系统	15
2.4 CA 和 TA 介绍	16
3 基于 OP-TEE 的应用开发	18
3.1 编写 CA 和 TA 所依赖的 GP API	18
3.2 示例程序	19
3.2.1 CA 文件	19
3.2.2 TA 文件	21
3.3 编译和运行	22
3.3.1 Yocto 域	22
3.3.2 Android 域	25
4 OP-TEE 的测试与调试	28
4.1 OP-TEE 的自测方式	28
4.2 OP-TEE 相关 Log 介绍	29
4.2.1 User Space	29
4.2.2 Linux Kernel	30
4.2.3 Secure World	30
4.3 OP-TEE 常见异常与分析	31

5 替换 OP-TEE TA 的签名与验签密钥 33

5.1 OP-TEE TA 签名验签简介 33

5.2 准备工作 33

5.3 替换 OP-TEE 镜像中的验证密钥 34

5.4 替换私钥 35

5.4.1 Yocto 侧替换私钥 35

5.4.2 Android 侧替换私钥 35

5.5 重新签名 TA 35

5.5.1 Yocto 侧重新签名 TA 36

5.5.2 Android 侧重新签名 TA 36

5.6 验证方式 37

6 附录 38

6.1 参考文档 38

附件一 附加条款 39

图片目录

图 1-1. 普通世界和可信世界 5

图 1-2. Arm 异常等级划分 6

图 1-3. 预留的 TEE OS memory 7

图 1-4. OP-TEE 架构 8

图 2-1. 非虚拟化下 OP-TEE 的组成 10

图 2-2. 虚拟化下 OP-TEE 的构成 11

图 2-3. 基于 OP-TEE 的安全存储 14

图 4-1. OP-TEE log 示例 31

表格目录

表 4-1. xtest 测项 28

表 5-1. MTK Android 测 TA 说明 36

1 概述

该文档介绍了 OP-TEE 的功能、结构设计、各组件的组成、如何编写 CA 和 TA，以及如果自测，如何 Debug。

本文档的章节安排为：

第 1 章为概述，介绍 TrustZone 的 OP-TEE 的基本概念。

第 2 章介绍 OP-TEE 的框架和功能。

第 3 章介绍如何开发基于 OP-TEE 的应用。

第 4 章为测试和调试，包含 OP-TEE OS 本身，以及 TA 的调试方式。

第 5 章介绍了如何替换 OP-TEE TA 的签名与验签密钥。

第 6 章为附录，列举了参考文件。

1.1 TrustZone 功能简介

TrustZone 是 Arm A-profile 架构中的安全架构名称。TrustZone 提供了两个执行环境，并在它们之间通过系统范围的硬件强制隔离，如 [Error! Reference source not found.](#)所示：

[Error! Reference source not found.](#)为普通世界和可信世界的示意图。普通世界运行一个丰富的软件栈。这个软件栈通常包括大量的应用程序集、一个复杂的操作系统（如 Linux），以及一个可能的虚拟机监控器。这样的软件栈庞大且复杂。尽管可以采取提高其安全性，但由于攻击面较大，它们更容易受到攻击。可信世界运行一个更小且更简单的软件栈，称为可信执行环境（TEE）。通常，TEE 包括由轻量级内核托管的几个可信服务。可信服务提供诸如密钥管理等功能。这个软件栈的攻击面显著较小，由于这个软件栈的攻击面显著较小，因此其脆弱性也大大减少。



图 1-1. 普通世界和可信世界

Armv8 和 v9 将 CPU 分为 4 种异常等级，分别是 EL0，EL1，EL2 和 EL3。

在 EL0、EL1 和 EL2 级别，处理器可以处于安全状态或非安全状态，这由 SCR_EL3.NS 位控制。通常会看到如下写法：

- NS-EL1: 非安全状态，异常级别 1
- S-EL1: 安全状态，异常级别 1
- EL3 始终处于安全状态，无论 SCR_EL3.NS 位的值如何。

安全状态和异常级别的安排如图 1-2 所示：

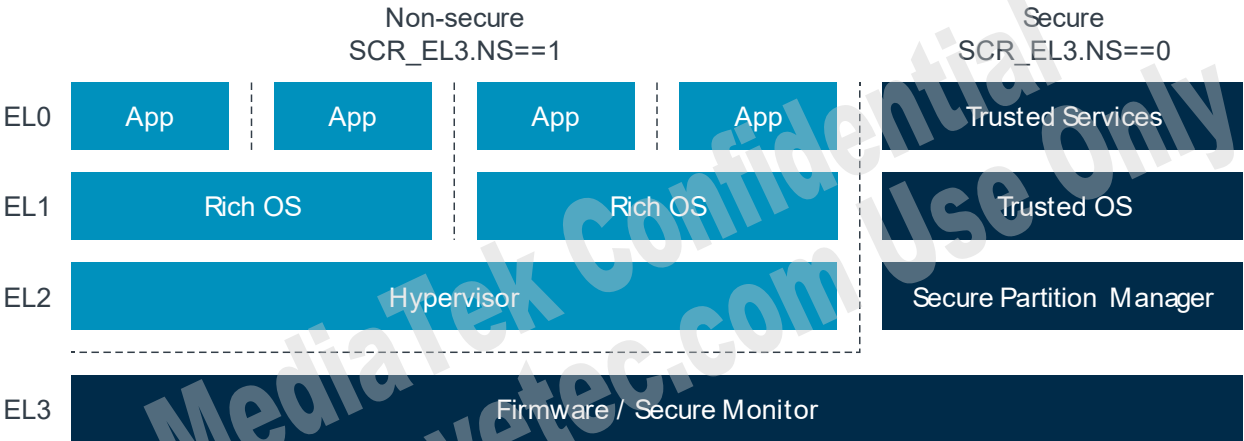


图 1-2. Arm 异常等级划分

Arm 为了保证 TrustZone 的安全性，提供了一系列的硬件隔离技术，包含中断隔离，片上 RAM/ROM 隔离，片外 RAM/ROM 隔离，外围设备的硬件隔离，外部 RAM 和 ROM 的隔离等。Arm 提供的隔离技术可以参阅 Arm 的技术文档，在此不再赘述。

MediaTek 提供了功能更加强大的内存保护单元 (SMPU) 保证 TrustZone 所在的内存区域只能被安全的 CPU 访问，阻止非安全的 CPU，以及各种外设通过各种方式访问 TrustZone 所在的内存区间。

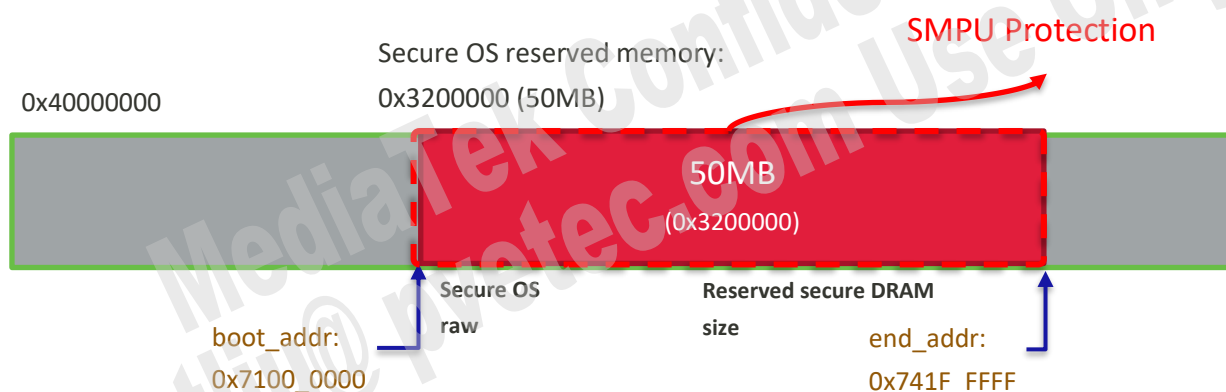


图 1-3. 预留的 TEE OS memory

图 1-3 是预留的 TEE memory 和 SMPU 保护这段 reserved memory 的示例图。这段 memory 只允许 Secure CPU 访问，即对应前文，只有 CPU 处在 S-EL0, S-EL1 或 EL3 时才可以访问；其他场景，例如 user space 程序，Linux Kernel 程序，hypervisor 程序都无权访问此段 memory。

1.2 OP-TEE 简介

OP-TEE 是一个开源的 Secure OS。图 1-4 是 OP-TEE 官方提供的架构图。

OP-TEE 内核是运行在 S-EL1 和 S-EL0 中的可信应用程序。可信应用程序通过 TEE 内部 API 与 OP-TEE 内核通信。TEE 内部 API 是由 GlobalPlatform 组织开发的标准 API。GlobalPlatform 致力于开发标准 API，这些 API 不仅支持 OP-TEE，还支持许多不同的 TEE。采用 GP 标准意味着在其他支持 GP 接口的 TEE 应用可以方便地移植到 OP-TEE 上，反之亦然，为 OP-TEE 设计的 TEE 应用也可以方便地移植到其他支持 GP 接口的 TEE 上。

在非安全状态下，内核空间中有一个 OP-TEE 驱动程序。它负责处理与 OP-TEE 内核的低级别通信。

在非安全用户空间（EL0）中，有一个用户空间库（libopenteec.so）实现了 GlobalPlatform API。TEE Client API 是应用程序用来访问 TA（可信应用程序）的接口。

OP-TEE 还包括一个称为 tee-supplciant 的组件。tee-supplciant 处理 OP-TEE 需要返回到 REE world 处理的事务，比如从文件系统中加载 UTA，再比如 TEE 安全存储功能需要把加密后的数据保存在 Flash 或者 RPMB 中，但 TEE 本身无法读写 flash 或 RPMB，需要切回 REE，通过 tee-supplciant 完成读写。

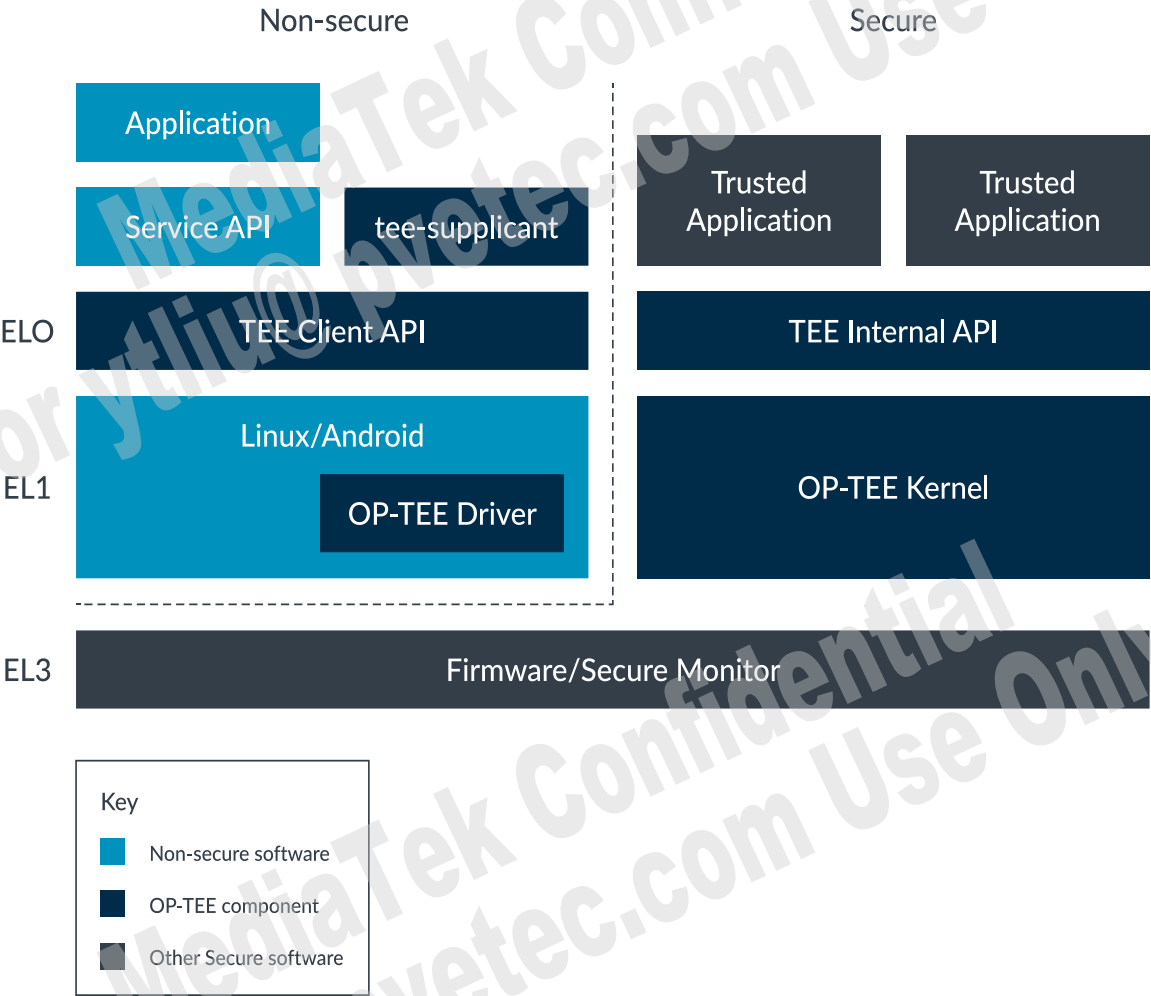


图 1-4. OP-TEE 架构

2 OP-TEE 框架和功能

2.1 开启虚拟化后 OP-TEE 核心组件

在[章节 1.2](#) 中我们简单提到了 OP-TEE 的构成，本章主要介绍开启虚拟化后 OP-TEE 的构成。

[图 2-1](#) 是未开启虚拟化时 OP-TEE 的组成，各组件分别为：

- Tee-supplciant：TEE 守护进程处理 load UTA，secure storage 读写等需要切回 REE 执行的事务
- libopenteec.so：提供 Client 端 GP API，CA 与 TA 通讯时使用
- OP-TEE Linux Driver：提供 shared memory 等 API，执行 SMC，与 Secure World 通讯
- OP-TEE OS：OPTEE Kernel 层提供的所有功能
- PTA（Pseudo Trusted Applications）：提供 CA 或 UTA 可以调用的 API，一些系统服务会以 PTA 的方式提供，PTA 运行在 OP-TEE Kernel 层，与 OP-TEE OS 打包在一起

基于 OP-TEE 的应用：CA & TA 为运行在 OP-TEE 上的应用

- CA（Client Agent）：运行在 REE（非安全世界）
- TA（Trusted App）：运行在 TEE（安全世界），需要受保护的操作和密钥可以放于 TA 中

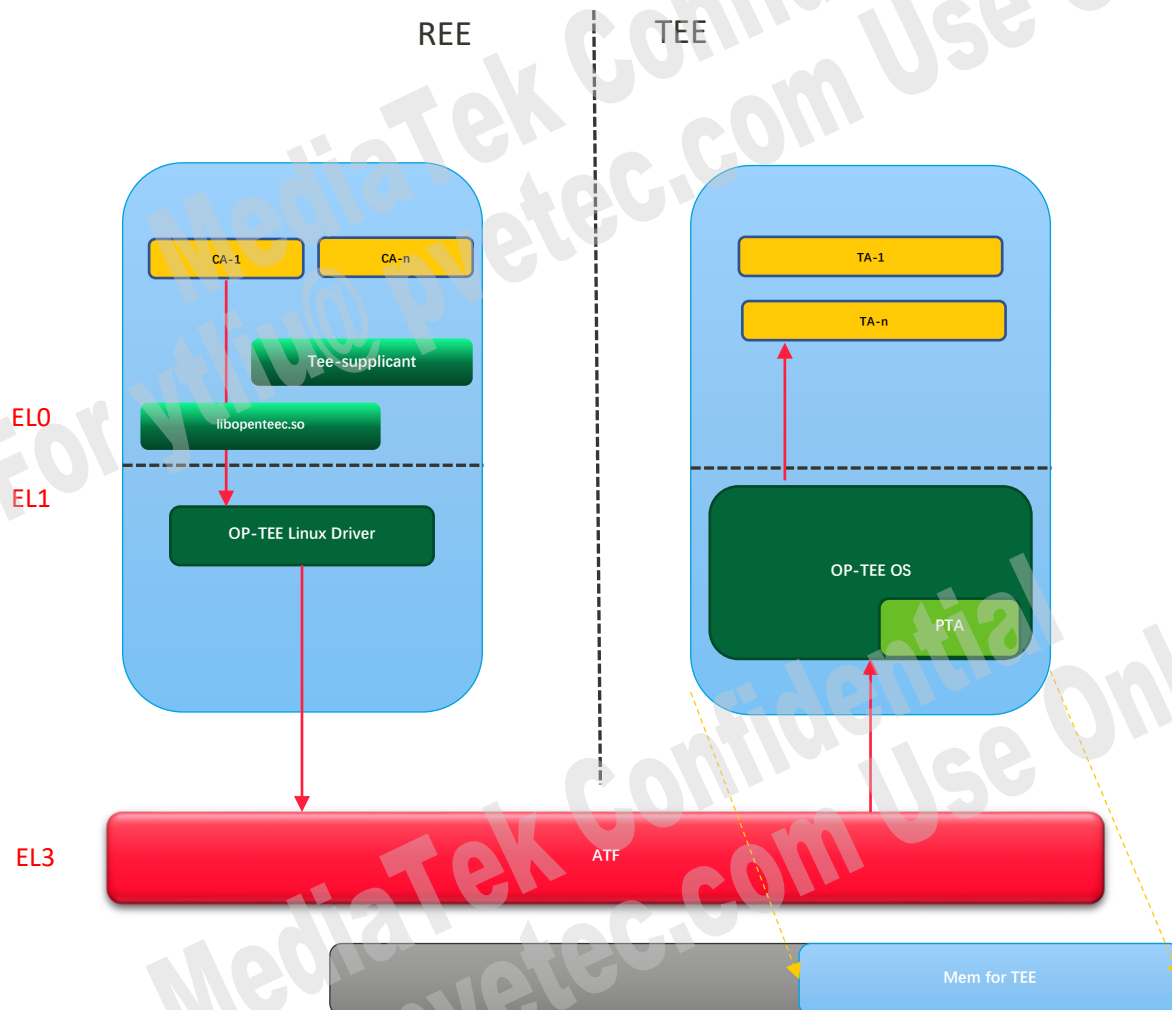


图 2-1. 非虚拟化下 OP-TEE 的组成

未开启虚拟化时，OP-TEE 的初始化在 ATF 跳转 BL32 时进行。

当开启 OP-TEE 虚拟化之后，OP-TEE 会在 TEE 内，为每一个 Host VM，创建一个对应的 OP-TEE 的 VM。

如图 2-2 所示，我们的 L+L+A 架构包含两个 Linux 的虚拟机和一个 Android 的虚拟机，分别为

- SOS (Linux 虚拟机)
- UOS (Android 虚拟机)
- UOS (Linux 虚拟机)

开启虚拟化后，ATF 跳转 BL32 做初始化时，只会初始化 OP-TEE nexus core 层。

在每个虚拟机创建之前，Hypervisor 层会发送 SMC command (OPTEE_SMC_VM_CREATED) 至 OP-TEE，创建出对应的 OP-TEE 虚拟机，并在预先分配好的安全内存中初始化 OP-TEE VM 所需的数据结构。

SOS (Linux 虚拟机) 的 TEE 请求会进入到 OP-TEE-VM1 中, OP-TEE-VM1 通过 RPC 或者 RET 返回到 REE 时, 也会回到 SOS 域

UOS (Android 虚拟机) 同理, 请求会进入到 OP-TEE-VM2, 从 TEE 返回时返回到 UOS (A)

UOS (Linux 虚拟机) 同理。

OP-TEE-VM1/VM2/VM3 在物理内存上是完全隔离的, 互不影响。

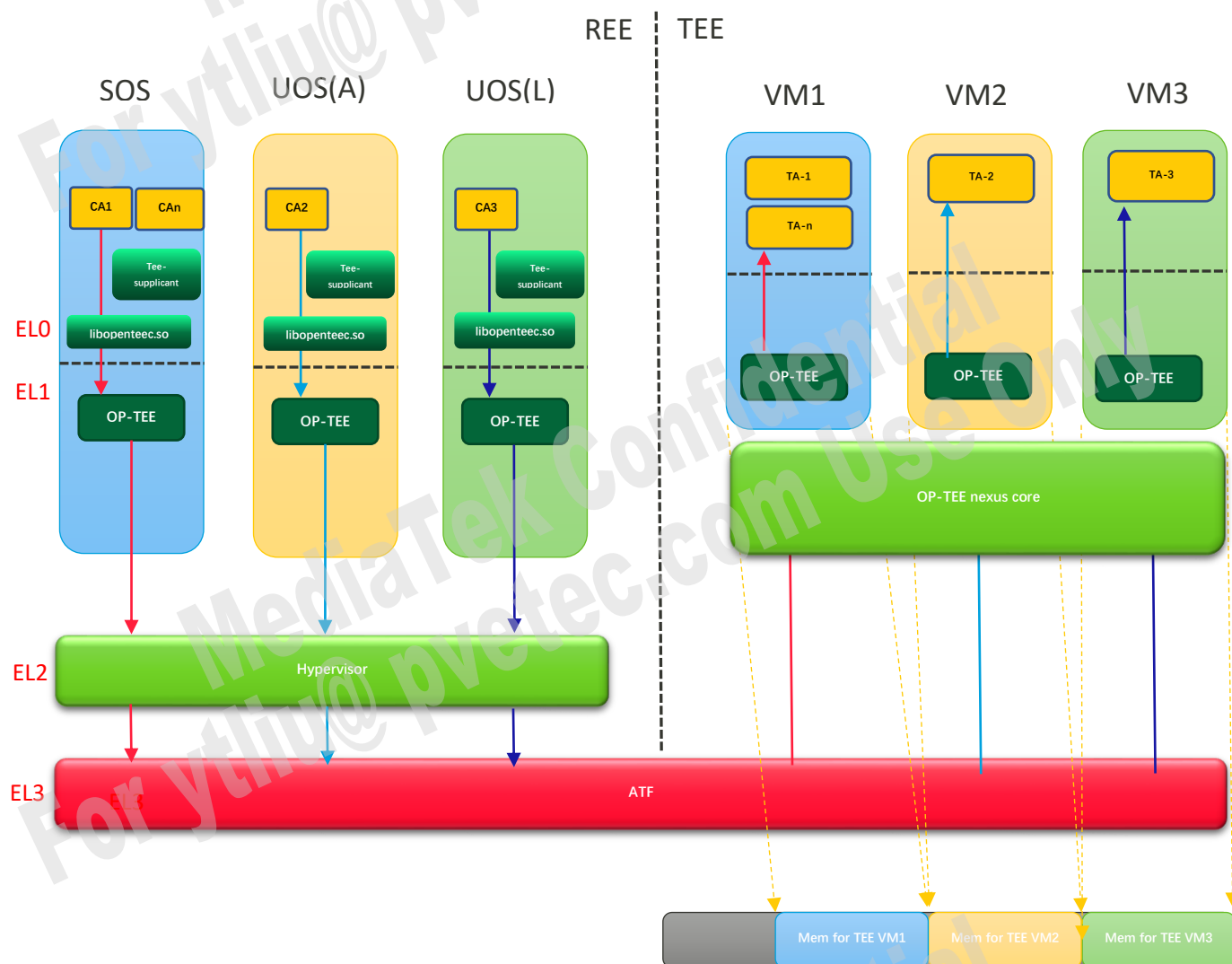


图 2-2. 虚拟化下 OP-TEE 的构成

2.2 MediaTek 平台 OP-TEE 开启方式

若需要开启或关闭 OP-TEE，需同时修改 Yocto 和 Android branch。下面分别介绍 Yocto branch 和 Android branch 需要做哪些改动。

2.2.1 Yocto Branch

请先检查如下目录存在；若如下目录不存在，则表示 release 不完全，需联系 MediaTek 处理。

- src/bsp/trustzone/optee/4.7.0/optee_client
- src/bsp/trustzone/optee/mtk_ext/client
- src/bsp/trustzone/optee/mtk_driver/secure_driver_spmlib
- src/bsp/trustzone/optee/mtk_ext/os_spmlib
- src/bsp/trustzone/optee/mtk_ext/plat_spmlib
- prebuilt/bsp/trustzone/optee/4.7.0/optee_os

确认一下目录都存在后，下面是 OP-TEE 的 config 配置方法：

Yocto branch 与 OP-TEE 相关的 Config 有两个，功能分别是：

TEE_SUPPORT：表示支援哪种 TEE，填入 “optee” 表示开启 OP-TEE。

TEE_TEST_SUPPORT：表示是否需要在编译时打包 OP-TEE 测试工具 xtest，此 config 只针对 OP-TEE 有效，默认为开启。

定义的位置在 Yocto 的 conf 文件，公版参考的路径为

meta/meta-mediatek-mt8668-hyp/conf/machine/{PROJECT_NAME}.conf
(PROJECT_NAME 需替换成您使用的 Yocto project，例如 auto8668p1_64_sos)

请客户根据自己的 project，选择对应的位置修改。

若需要开启 OP-TEE，请将 config 设定为：

```
TEE_SUPPORT = "optee"
```

若需要关闭 OP-TEE，请将 config 设定为：

```
TEE_SUPPORT = "none"
```

若需要开启 OP-TEE，请将 Kernel dts 中 OP-TEE 节点的 disabled status 删除或者改为 okay status：

```
optee {  
    status = "okay";  
}
```

2.2.2 Android Branch

请先检查如下目录存在；若如下目录不存在，则表示 release 不完全，需联系 MediaTek 处理

- vendor/mediatek/proprietary/trustzone/optee/4.7.0/optee_client
- vendor/mediatek/proprietary/trustzone/optee/mtk_ext/client
- vendor/mediatek/proprietary/trustzone/optee/mtk_driver/secure_driver_spmlib
- vendor/mediatek/proprietary/trustzone/optee/mtk_ext/os_spmlib
- vendor/mediatek/proprietary/trustzone/optee/mtk_ext/plat_spmlib
- vendor/mediatek/proprietary/trustzone/optee/4.7.0/optee_os_spmlib

Android branch 与 OP-TEE 相关的 config 有两个，分别的功能是

MTK_OPTEE_SUPPORT: Vendor 层是否打开 OP-TEE，若打开，则会把 OP-TEE 的 binary 打包进 tee.img。

MGVI_MTK_OPTEE_SUPPORT: HAL 层 config，此 config 不会影响 OP-TEE 的开启或关闭，但 Android 域基于 OP-TEE 的 feature 可能会基于这个 config 判断自身的开启和关闭，故建议一并开启。

开启的位置为：

MTK_OPTEE_SUPPORT 在 ProjectConfig.mk 中

MediaTek 公版示例文件路径为：

device/mediateksample/{PROJECT_NAME}/ProjectConfig.mk

(PROJECT_NAME 需替换成您使用的 Android project，例如 auto8668p1_64_vm_uos)

MGVI_MTK_OPTEE_SUPPORT 定义在 VendorConfig.mk 中，

MediaTek 公版示例文件路径为：

device/mediatek/vendor/{PROJECT_NAME}/VendorConfig.mk

(PROJECT_NAME 需替换成您使用的 Android mgvi project，例如 mgvi_auto_64_armv82_wifi_vm_uos)

若开启 OP-TEE，则将这两个 config 值为 yes

```
MTK_OPTEE_SUPPORT = yes
MGVI_MTK_OPTEE_SUPPORT = yes
```

若需要关闭 OP-TEE，将这两个 config 置为 no，或者直接删除这两个 config 的定义皆可。

Kernel 部分所需修改与 Yocto 相同，请参考上节内容，用同样的方法打开 Android 域 Kernel 层 OP-TEE。

2.3 安全存储机制

在 OP-TEE 中，安全存储是根据 GlobalPlatform 的 TEE Core API 定义实现的。该规范要求必须能够存储通用数据和密钥材料，并保证存储数据的机密性和完整性，以及修改存储的操作的原子性（这里的原子性意味着整个操作要么成功完成，要么不进行任何写入）。

目前在 OP-TEE 中有两种安全存储实现。

2.3.1 REE 文件系统

使用外部（非安全）文件系统来存储加密的数据。这种方式依赖于正常世界（Normal World）的文件系统，但所有的敏感操作（如加密和解密）都在安全世界（Secure World）内完成。其实现原理请参阅图 2-3。

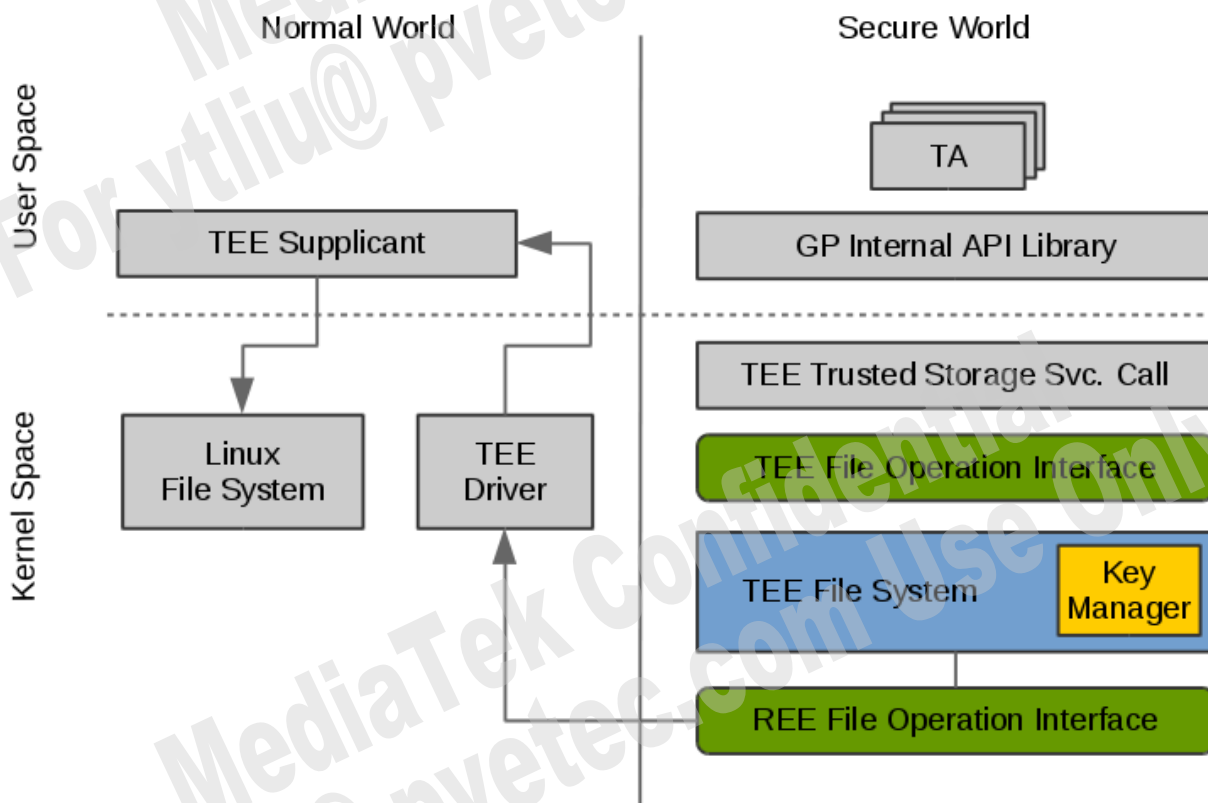


图 2-3. 基于 OP-TEE 的安全存储

OP-TEE 默认支持此种方式的安全存储。

注意：

- 使用 REE FS 形式的安全存储保存的数据，会保存在对应 VM 的文件系统中，用户或程序无法在 REE 测读取到文件的内容，但有读写权限的用户或程序可能篡改或删除此文件。

Yocto 域默认存储位置为：`/var/lib/tee` 或 `/data/vendor/tee`

若想修改此路径，方式为：

Yocto branch 上找到

`meta/meta-mediatek/recipes-bsp/trustzone/optee-client_4.7.0.bb`

修改 `CFG_TEE_FS_PARENT_PATH` 的定义。

Android 域默认存储路径为：/data/vendor/tee

若想修改此路径，方式为：

Android branch 上找到

vendor/mediatek/proprietary/trustzone/optee/4.7.0/optee_clent/Android.mk

修改 CFG_TEE_FS_PARENT_PATH 的定义。

注意：

- 请读者根据需求修改此目录，修改时请留意 tee-supplciant 是否有权限创建目录；目录所在分区是否会因重启或者恢复出厂设置而被重置。

2.3.2 RPMB 文件系统

基于 RPMB 的安全存储，数据加解密方式与 REE FS 方式类似，只是存储介质由普通文件系统变为存储在 UFS（或 eMMC）附带的 RPMB 区域，利用 RPMB 的防篡改和防重放特性，提供更高的安全性。存储原理与图 2-3 类似，只是将左下角的“Linux File System”替换成 RPMB 的 driver。

若想在虚拟化的项目上支持基于 RPMB 的安全存储，需要 UFS 硬件支持多个 RPMB region（例如需要 UFS3.0 及以上版本，eMMC 不支持），因此此种存储方式并不是默认支持的，需联系 MediaTek 确认此方式在您的硬件上是否支持。

在编写 TA 程序时，需通过参数指定，将数据存储与 REE 文件系统和 RPMB 文件系统实现。

定义了两个 OP-TEE 特定的存储标识符：TEE_STORAGE_PRIVATE_REE 和 TEE_STORAGE_PRIVATE_RPMB。

TEE_STORAGE_PRIVATE_REE 表示将数据保存于 REE 文件系统；

TEE_STORAGE_PRIVATE_RPMB 表示将数据保存于 RPMB 文件系统。

注意：

- 无论是 REE FS 还是 RPMB 存储方式，各个 VM 存储的数据不互通；即由哪个域写入，就只能从哪个域读取。

在平台上执行 xtest 6001 测项，可以判断中平台上安全存储功能是否正常。

```
* regression_6001 Test TEE_CreatePersistentObject
o regression_6001.1 Storage id: 00000001
  regression_6001.1 OK
o regression_6001.2 Storage id: 80000000 -> REE FS 方式的安全存储
  regression_6001.2 OK
o regression_6001.3 Storage id: 80000100 -> RPMB 方式的安全存储
  regression_6001.3 OK
  regression_6001 OK
```

若执行 xtest 6001 测试时，没有执行 regression_6001.3 测项，则说明当前设备并不支持 RPMB，不支持的原因可能是 RPMB key 未烧录，或软硬件环境不满足需求；若确定有 RPMB 需求，请联系 MediaTek 做进一步排查。有关 xtest 如何执行，可以参阅本文[章节 4.1](#)。

2.4 CA 和 TA 介绍

在 OP-TEE 中，CA 通常指的是"Client Application"（客户端应用程序）。CA 是运行在普通世界（REE，Rich Execution Environment）中的应用程序，它与运行在受信世界（TEE，Trusted Execution Environment）中的受信应用程序（TA，Trusted Application）进行通信。CA 通过调用 TEE API 来请求 TA 执行安全操作。

以下是 CA 和 TA 之间的基本交互流程：

1. CA 启动：客户端应用程序在普通世界中启动。
2. 初始化 TEE 会话：CA 通过 TEE API 初始化与 TA 的会话。
3. 发送命令：CA 向 TA 发送命令请求，要求 TA 执行特定的安全操作。
4. TA 处理请求：TA 在受信世界中处理请求，并返回结果。
5. 接收结果：CA 接收 TA 返回的结果，并根据需要进行处理。
6. 关闭会话：操作完成后，CA 关闭与 TA 的会话。

用户可根据需求，在 user space 或 Kernel space 编写 CA。

在 OP-TEE 中，TA 指 "Trusted Application"（可信应用）。OP-TEE 支持三种类型的可信应用（TA），分别是用户模式可信应用（UTA）、静态可信应用（PTA，也称为 Pseudo TA）和早期用户模式可信应用（Early UTA）。这些应用类型在功能、执行时机和运行环境方面有所不同。下面是它们的主要区别：

1. 用户模式可信应用（UTA）
 - 运行环境：UTA 在用户模式下运行，与操作系统的内核模式相对立。这意味着它们在一个隔离的环境中执行，提供了更高的安全性。
 - 部署方式：UTA 通常作为单独的应用程序部署，可以通过安全存储进行加载和管理。
 - 用途：UTA 适用于处理高度敏感的数据或执行安全关键的任务，如加密操作、密钥管理等。
2. 静态可信应用（PTA）
 - 运行环境：PTA 在 TEE 内核模式下运行，与 UTA 相比，它们有更高的权限和更直接的访问 TEE 核心服务的能力。
 - 部署方式：PTA 通常是在 TEE 操作系统构建时静态集成的，不是作为独立应用程序加载的。

- 用途：PTA 用于实现一些需要更高权限访问 TEE 资源的功能，如管理和控制 TEE 内部的其他组件或服务。
3. 早期用户模式可信应用（Early UTA）
- 运行环境：Early UTA 也在用户模式下运行，但它们在系统启动的早期阶段被加载和执行。
 - 部署方式：与普通的 UTA 相似，但它们在 TEE 初始化过程中较早启动，用于处理启动早期阶段的安全任务。
 - 用途：Early UTA 通常用于执行系统启动过程中的安全检查或配置，如验证启动配置的完整性或早期硬件资源的安全配置。

注意：

- 目前基于 MediaTek 平台的 OP-TEE 只支持开发 UTA，若有 PTA 或 Early UTA 的开发需求，请联系 MediaTek。

UTA 会放置在文件系统上，

Yocto 域的路径是 `/lib/optee_armtz/`。

Android 域的路径是 `/vendor/lib/optee_armtz/`。

若一个 TA 在 Yocto 域和 Android 域都需要，您需要在每个域的文件系统上都放置一份。

3 基于 OP-TEE 的应用开发

在 Yocto 域和 Android 域都支持 OP-TEE，且 Yocto 域和 Android 域所需的 API 都是一样的。

这意味着 Yocto 域和 Android 域的 CA/TA 代码可以完全一样。

如果您的程序需要运行在 Yocto 域，需要将 CA 和 TA 的执行文件打包进 Yocto 域的文件系统，

Android 域同理，若需要运行于 Android 域，也需要将执行文件打包进 Android 域的文件系统。

3.1 编写 CA 和 TA 所依赖的 GP API

OP-TEE 支持 GP (GlobalPlatform) API，意味着您的 CA/TA 程序可以方便的在其他支持 GP API 的 TEE OS 上进行快速移植。关于 GP API 的资料，您可以自行查阅互联网上的资料。

CA 侧使用的 GP API 以 TEEC 开头，您可以在如下头文件中找到相关定义

`${PATH}/optee/4.7.0/optee_client/libtee/include/tee_client_api_extensions.h`

其中，Yocto branch 上的路径为：`src/bsp/trustzone`

Android branch 上的路径为：`vendor/mediatek/proprietary/trustzone`

CA 端负责与 TEE 通讯的 API 主要是以下五个 API：

1. `TEEC_Result TEEC_InitializeContext(
 const char* name,
 TEEC_Context* context)`
2. `void TEEC_FinalizeContext(
 TEEC_Context* context)`
3. `TEEC_Result TEEC_OpenSession (
 TEEC_Context* context,
 TEEC_Session* session,
 const TEEC_UUID* destination,
 uint32_t connectionMethod,
 const void* connectionData,
 TEEC_Operation* operation,
 uint32_t* returnOrigin)`
4. `void TEEC_CloseSession (
 TEEC_Session* session)`
5. `TEEC_Result TEEC_InvokeCommand(
 TEEC_Session* session,
 uint32_t commandID,
 TEEC_Operation* operation,
 uint32_t* returnOrigin)`

CA 端与共享内存操作有关的 API 是如下三个：

1. `TEEC_Result TEEC_RegisterSharedMemory(TEEC_Context *context, TEEC_SharedMemory *sharedMem);`
2. `TEEC_Result TEEC_AllocateSharedMemory(TEEC_Context *context, TEEC_SharedMemory *sharedMem);`
3. `void TEEC_ReleaseSharedMemory(TEEC_SharedMemory *sharedMemory);`

GP 在 TEE 内的 API 以 “TEE_” 开头，涉及到 TEE 内的各种操作，例如加解密操作，安全文件读写等，请参阅 GP 相关文档，这里不再赘述。

编写一个 TA，必须实现以下五个 API。

1. `TEE_Result TA_CreateEntryPoint(void)`
2. `void TA_DestroyEntryPoint(void)`
3. `_Result TA_OpenSessionEntryPoint(uint32_t param_types, TEE_Param __maybe_unused params[4], void __maybe_unused **sess_ctx)`
4. `void TA_CloseSessionEntryPoint(void __maybe_unused *sess_ctx)`
5. `TEE_Result TA_InvokeCommandEntryPoint(void __maybe_unused *sess_ctx, uint32_t cmd_id, uint32_t param_types, TEE_Param params[4])`

接下来我们结合一个示例程序讲解如何编写一个最简单的 CA 和 TA。

3.2 示例程序

MediaTek 撰写了一份 sample code，讲解如何写一个简单的 demo 程序，这份 sample code 可同时用于 Yocto 域和 Android 域。如果有需要全部源码文件，可联系 MediaTek 获取。

本章会基于 MediaTek 的 demo code 做讲解。

3.2.1 CA 文件

CA 部分的开发主要是对章节 3.1 介绍的 TEE Client API 的使用，包括如何初始化上下文，如何打开/关闭会话，以及如何使用不同的参数类型进出 TA。可参考该 demo 的调用流程进行 CA 的开发。

如下是初始化、销毁 TEE 上下文和打开、关闭会话的流程，如果没有多实例 TA 并发的需求，建议参考 demo，使用全局变量控制上下文和会话数量为单个。

```
static TEEC_Session g_stTeecSession = {0};
```

```

static TEEC_Context g_stTeecContext = {0};
static bool g_bIsTeecInitialized = false;

TEEC_Result initTzContext()
{
    const TEEC_UUID stTeecUuid = {0xa60589fd, 0x0158, 0x40eb, {0xa7, 0x74, 0xa3, 0x98,
0xb1, 0xd6, 0x98, 0x32}};
    TEEC_Result u32TeecResult = TEEC_ERROR_GENERIC;
    uint32_t u32RetOrig = 0;

    if (true == g_bIsTeecInitialized)
    {
        u32TeecResult = TEEC_SUCCESS;
        goto exit;
    }

    u32TeecResult = TEEC_InitializeContext(NULL, &g_stTeecContext);
    if (TEEC_SUCCESS != u32TeecResult)
    {
        DLOGE("TEEC_InitializeContext fail, %X", u32TeecResult);
        goto exit;
    }

    u32TeecResult = TEEC_OpenSession(&g_stTeecContext, &g_stTeecSession,
&stTeecUuid, TEEC_LOGIN_PUBLIC, NULL, NULL, &u32RetOrig);
    if (TEEC_SUCCESS != u32TeecResult)
    {
        TEEC_FinalizeContext(&g_stTeecContext);
        DLOGE("TEEC_OpenSession fail, %X, orig: %X", u32TeecResult, u32RetOrig);
        goto exit;
    }

    g_bIsTeecInitialized = true;
exit:
    return u32TeecResult;
}

TEEC_Result uninitTzContext()
{
    TEEC_Result u32TeecResult = TEEC_ERROR_GENERIC;

    if (false == g_bIsTeecInitialized)
    {
        u32TeecResult = TEEC_ERROR_GENERIC;
        DLOGE("Session not exist!");
        goto exit;
    }

    TEEC_CloseSession(&g_stTeecSession);
    TEEC_FinalizeContext(&g_stTeecContext);

    u32TeecResult = TEEC_SUCCESS;
    g_bIsTeecInitialized = false;
}

```

```

exit:
    return u32TeecResult;
}

```

当 CA 发送命令到 TA 时，需要使用 TEEC_InvokeCommand 函数，该函数支持 CA 带不同类型的参数到 TA，如下以值类型为例展示该函数的使用方法。

```

TEEC_Result testValueParameter()
{
    TEEC_Result    u32TeecResult    = TEEC_ERROR_GENERIC;
    uint32_t       u32RetOrig       = 0;
    TEEC_Operation stTeecOperation = {0};

    memset(&stTeecOperation, 0, sizeof(TEEC_Operation));
    stTeecOperation.paramTypes = TEEC_PARAM_TYPES(TEEC_VALUE_INPUT,
        TEEC_NONE,
        TEEC_NONE,
        TEEC_NONE);
    stTeecOperation.params[0].value.a = 0x11223344;
    stTeecOperation.params[0].value.b = 0x55667788;
    u32TeecResult = TEEC_InvokeCommand(&g_stTeecSession, E_DEMO_CMD_VALUE_INPUT_TEST,
    &stTeecOperation, &u32RetOrig);
    if ((TEEC_SUCCESS != u32TeecResult) || (TEEC_ORIGIN_TRUSTED_APP != u32RetOrig))
    {
        DLOGE("TEEC_InvokeCommand fail, result=%X, origin=%X", u32TeecResult, u32RetOrig);
        goto exit;
    }

exit:
    return u32TeecResult;
}

```

3.2.2 TA 文件

TA 部分实现了基于不同的命令标识来执行 CA 请求的各种操作，TA 的开发要定义自己的 UUID，在 CA 调用 TEEC_OpenSession 时传入。同时该 demo 也在 user_ta_header_defines.h 中将 TA 定义为单实例，即 TA 的二进制在 OP-TEE 内存中只会存一份，不会多次加载。TA_STACK_SIZE 和 TA_DATA_SIZE 则需要根据各 TA 的实现自行调整大小。

```

#define TA_UUID { 0xa60589fd, 0x0158, 0x40eb, {0xa7, 0x74, 0xa3, 0x98, 0xb1, 0xd6, 0x98, 0x32} }

TEEC_Result TA_CreateEntryPoint(void)

#define TA_FLAGS (TA_FLAG_SINGLE_INSTANCE | \
TA_FLAG_MULTI_SESSION \
)

#define TA_STACK_SIZE (2 * 1024)

```

```
#define TA_DATA_SIZE (32 * 1024)
```

当 TA 收到 CA 发送的命令时，即可根据命令标识来进行参数检查，并进行相应的功能实现。

```
TEE_Result TA_InvokeCommandEntryPoint(void *pSessionContext, uint32_t nCommandID, uint32_t
nParamTypes, TEE_Param pParams[4])
{
    (void)pSessionContext;
    DLOGE("[TA]CommandID :%d \n", nCommandID);
    switch (nCommandID)
    {
        case E_DEMO_CMD_NO_PARA_TEST:
            (void)pParams;
            return demo_ta_no_para_test(nParamTypes);
        case E_DEMO_CMD_VALUE_INPUT_TEST:
            return demo_ta_value_input_test(nParamTypes,pParams);
        case E_DEMO_CMD_VALUE_OUTPUT_TEST:
            return demo_ta_value_output_test(nParamTypes,pParams);
        case E_DEMO_CMD_VALUE_INOUT_TEST:
            return demo_ta_value_inout_test(nParamTypes,pParams);
        case E_DEMO_CMD_MEMREF_TEMP_INPUT_TEST:
            return demo_ta_memref_temp_input_test(nParamTypes,pParams);
        case E_DEMO_CMD_MEMREF_TEMP_OUTPUT_TEST:
            return demo_ta_memref_temp_output_test(nParamTypes,pParams);
        case E_DEMO_CMD_MEMREF_TEMP_INOUT_TEST:
            return demo_ta_memref_temp_inout_test(nParamTypes,pParams);
        default:
            DLOGE("Unexpected command id: %d", nCommandID);
            return TEE_ERROR_BAD_PARAMETERS;
    }
}
```

3.3 编译和运行

CA 的编译需要依赖库文件 `libopentec.so`；TA 的编译需要依赖 MediaTek 提供的开发包。

Android 域的开发包会放在 `vendor/mediatek/proprietary/trustzone/optee/4.7.0/optee_os_spmllib` 路径。

Yocto 域的开发包放在 `prebuilt/bsp/trustzone/optee/4.7.0/optee_os` 路径。

3.3.1 Yocto 域

Yocto 域 CA/TA 的编译建议通过 Yocto 原生方式，可以参考如下写法：

```
inherit fog
inherit deploy
```

```
DESCRIPTION = "demo_tz_app"
LICENSE = "MediaTekProprietary"
```

```

LIC_FILES_CHKSUM =
"file://${PROPRIETARY_LICENSE_PATH}/MediaTekProprietary;md5=c5d17c6905715d0948a3d6087602d12d
"

SRC_URI = "fog://src/bsp/tzapp2/demo_tz_app;name=demo_tz_app"
PACKAGES = "${PN}"
INHIBIT_PACKAGE_DEBUG_SPLIT = "1"

SRCREV_demo_tz_app = "${AUTOREV}"
S = "${WORKDIR}/git"

DEPENDS += "optee-client"
DEPENDS += "optee-os-tadevkit"

BUILD_FROM_YOCTO ?= "true"
TA_DEV_KIT_DIR = "${STAGING_INCDIR}/optee/export-user_ta"

EXTRA_OEMAKE = " \
    CROSS_COMPILE=${TARGET_PREFIX} \
    TA_DEV_KIT_DIR=${TA_DEV_KIT_DIR} \
    BUILD_FROM_YOCTO=${BUILD_FROM_YOCTO} \
"
EXTRA_OEMAKE:append = "LIBGCC_LOCATE_CFLAGS=--sysroot=${STAGING_DIR_HOST}"

INSANE_SKIP:${PN} += "already-stripped"
FILES:${PN} += "${libdir}"
FILES:${PN} += "${bindir}"
FILES:${PN} += "${nonarch_base_libdir}/optee_armtz/"

do_compile() {
    if [ -e makefile ]; then
        oe_runmake
    fi
}

do_install () {
    if [ -e makefile ]; then
        oe_runmake O="${WORKDIR}" DESTDIR="${D}" install
    fi
}

```

在 src/bsp/tzapp2/demo_tz_app 中，提供 makefile 来给 bb 文件调用

```

.PHONY: all clean install

all:
    $(MAKE) -C demoCA setup
    $(MAKE) -C demoCA build BUILD_FROM_YOCTO=${BUILD_FROM_YOCTO}
    $(MAKE) -C demoTA TA_DEV_KIT_DIR=${TA_DEV_KIT_DIR}
    LIBGCC_LOCATE_CFLAGS=${LIBGCC_LOCATE_CFLAGS}

clean:
    $(MAKE) -C demoCA clean
    $(MAKE) -C demoTA clean

```

```

install:
    install -d $(DESTDIR)/usr/bin
    install -m 755 demoCA/out/demoTzApp $(DESTDIR)/usr/bin
    install -d ${DESTDIR}${nonarch_base_libdir}/optee_armtz
    install -m 0444 demoTA/out/*.ta ${DESTDIR}${nonarch_base_libdir}/optee_armtz

uninstall:
    rm -f $(DESTDIR)/usr/bin/demoTzApp
    rm -f ${DESTDIR}${nonarch_base_libdir}/optee_armtz/${BINARY}.ta

```

CA 的 makefile 参考如下

```

TOP_DIR = ${CURDIR}
OBJ_PATH = ./out
TARGET := demoTzApp

SRC += $(TOP_DIR)/src/main.c
INC += -I./include
LIBS += -lopentec

CFLAGS := $(INC) \
        -Werror \
        -fPIC -g -O2 \
        -fstack-protector

ifeq ($(BUILD_FROM_YOCTO), true)
CFLAGS += -DANDROID_BUILD=0
endif

OBJS := $(SRC:.c=.o)
OBJS_S := $(SRC_S:.S=.o)

.PHONY: all

all: clean setup build

setup:
    @mkdir -p $(OBJ_PATH)

build: $(OBJS) $(OBJS_S)
    $(CC) $(CFLAGS) $(LDFLAGS) -o $(OBJ_PATH)/$(TARGET) $(OBJ_PATH)/*.o $(LIBS)

$(OBJS): %.o: %.c
    $(CC) $(CPPFLAGS) $(CFLAGS) -o $(OBJ_PATH)/$(notdir $@) -c $<

$(OBJS_S): %.o: %.S
    $(CC) $(CPPFLAGS) $(CFLAGS) -o $(OBJ_PATH)/$(notdir $@) -c $<

clean:
    @rm -rf $(OBJ_PATH)

```

TA 的编译需要同时提供 makefile 和 sub.mk，makefile 参考如下

```
ifneq ($O,)
out-dir := $O
else
out-dir := $(CURDIR)/out
endif
O=$(out-dir)

BINARY = a60589fd-0158-40eb-a774-a398b1d69832
export BINARY
override LDADD += -L$(O)/../

-include $(TA_DEV_KIT_DIR)/mk/ta_dev_kit.mk

ifeq ($(wildcard $(TA_DEV_KIT_DIR)/mk/ta_dev_kit.mk), )
clean:
    @echo 'Note: $$ (TA_DEV_KIT_DIR)/mk/ta_dev_kit.mk not found, cannot clean TA'
    @echo 'Note: TA_DEV_KIT_DIR=$(TA_DEV_KIT_DIR) '
endif
```

sub.mk 参考如下

```
global-incdirs-y += include
srcs-y += src/demo_ta_entry.c
srcs-y += src/demo_ta_commands.c
```

编译命令参考：

```
source meta/poky/oe-init-build-env && bitbake demo-tz-app -c cleansstate; bitbake demo-tz-app 2>&1 | tee mtk_demo.log
```

该命令会同时编译出 CA & TA 档案。

可于如下路径查看编译产物：

Yocto CA:

- build/tmp/work/aarch64-poky-linux/demo-tz-app/1.0/git/demoCA/out/demoTzApp

Yocto TA:

- build/tmp/work/aarch64-poky-linux/demo-tz-app/1.0/git/demoTA/out/a60589fd-0158-40eb-a774-a398b1d69832.ta

将 TA (a60589fd-0158-40eb-a774-a398b1d69832.ta) push 进平台的/lib/optee_armtz/路径下；

将 CA (demoTzApp) push 进平台的任意可写路径，修改权限为可执行后，即可执行 demo 程序。

3.3.2 Android 域

Android 域的 CA 建议使用 Android 自带的编译系统，Android.bp 写法参考如下：

```
cc_binary {
    name: "demoTzApp",
    vendor: true,
    owner: "mtk",
    proprietary: true,
    cflags: ["-DANDROID_BUILD=1"],
    local_include_dirs: ["include"],
    srcs: ["src/main.c"],
    shared_libs: ["liblog", "libopenteec"],
    compile_multilib: "both",
}
```

以 MT8668 平台为例，编译命令为：

```
source build/envsetup.sh && export OUT_DIR=out_demo && lunch
hal_mgvi_auto_64_armv82_wifi_vm_uos-next-userdebug && mmm
vendor/mediatek/proprietary/trustzone/demo_tz_app/demoCA | tee make_demoCA.log
```

Android 域 TA 编译使用脚本，参考如下：

```
export TA_DEV_KIT_DIR=$PWD/export-ta_arm64
if [ -d ${TA_DEV_KIT_DIR} ]; then
    echo "TA_DEV_KIT_DIR = ${TA_DEV_KIT_DIR}"
else
    tar xzf tz_optee_4.7.0_dev_kit.tar.gz
    echo -e "\033[1;32m ##### uncompress: TA_DEV_KIT_DIR = ${TA_DEV_KIT_DIR}. #####\033[m"
fi

#### !!! Need to modify ANDROID_TOP !!! ###
ANDROID_TOP=/xxx/alps-mp-xxx-20xx_xx_xx_xx_xx
export PATH=${ANDROID_TOP}/prebuilts/clang/clang-tee/linux-x86/bin/${PATH}
export COMPILER=clang
export O=out

make clean
make
```

将 MediaTek 提供的 Android OP-TEE TA 开发包和[章节 3.3.1](#)提供的 TA 编译的 makefile 和 sub.mk 放到同一级路径下即可进行编译。

注意：

- 使用该脚本前，需要自行设定 ANDROID_TOP 值为 MediaTek Android 基线的路径

可于如下路径查看编译产物：

Android CA:

- out_demo/target/product/mgvi_auto_64_armv82_wifi_vm_uos/vendor/bin/demoTzApp

Android TA:

- vendor/mediatek/proprietary/trustzone/demo_tz_app/demoTA/out/a60589fd-0158-40eb-a774-

a398b1d69832.ta

将 TA (a60589fd-0158-40eb-a774-a398b1d69832.ta) push 进平台的 `/vendor/lib/optee_armtz/` 路径下;

将 CA (demoTzApp) push 进平台的任意可写路径, 修改权限为可执行后, 即可执行 demo 程序。

4 OP-TEE 的测试与调试

4.1 OP-TEE 的自测方式

OP-TEE 官方有搭配自测工具，名为 xtest。

在 Yocto 域，MediaTek 会释放 xtest 代码，可编译 xtest 并打包进 load 中；在 Android 域，因 release policy 的原因，MediaTek 未提供 xtest 源码，需要 push binary 文件进平台中进行测试。

xtest 可以通过查阅源码观察测项内容，这里简单整理如表 4-1 所示：

表 4-1. xtest 测项

Xtest 测项	测试目的简介
xtest_1	OP-TEE 基本功能测试，如进出 TEE 的测试，TA 的 panic 测试等
xtest_2	socket 相关测试
xtest_4	加解密算法测试
xtest_5	Share memory 测试
xtest_6	安全存储相关测试
xtest_8	KDF, Mbed 等杂项测试
Pkcs11	PKCS11 相关组件测试

在 Yocto 域 xtest 由如下文件组成：

/usr/bin/xtest -> xtest 的可执行程序
/lib/optee_armtz/*.ta -> xtest 测试依赖的 UTA
在串口或 adb shell 中输入 “xtest” 即可开始执行 xtest 测试

在 Android 域 xtest 由如下文件组成：

/vendor/bin/xtest -> xtest 的可执行程序
/vendor/lib/optee_armtz/*.ta -> xtest 测试依赖的 UTA

Android 域执行 xtest 前需要用 “su” 切换到管理员权限，然后 setenforce 0 关闭 selinux，然后执行 xtest

```
# su
# setenforce 0
# xtest
```

待 xtest 执行完毕后，会打印出有多少测项 pass 和 fail。

若 xtest 无法执行，则说明当前环境 OP-TEE 异常，请先按照如下方式自查：

1. 查看 optee.ko 和 tee.ko 是否有正确 insmod 进去

```
# lsmod | grep tee
```

正常情况下，应看到 TEE 和 OP-TEE driver 被正确加载。

2. 查看 tee-supplciant 进程是否有运行起来

通过 adb 或串口进入 shell，输入以下命令

```
# ps -A | grep tee
```

查看 tee-supplciant 进程是否存在

正常情况下，tee-supplciant 进程在后台运行，例如

```
sh-3.2# ps -A | grep tee
2250 ?          00:00:00 tee-supplciant
```

```
$ ps -A | grep tee
root          1125      1      2176420   10168  0          0 S tee-supplciant
```

4.2 OP-TEE 相关 Log 介绍

通过前面的介绍，函数从 CA 跳转入 TA，一般需要通过如下几个部分

CA -> libopentee.so -> linux kernel -> ATF -> optee os -> TA

我们介绍 log 的时候，也会分别介绍这几个模块中的 log 会如何打印。

Libopentee.so/linux kernel/ATF 中的逻辑较为简单，一般不会遇到问题，最常见的问题出现在 CA、TA 以及 OP-TEE OS 中。

4.2.1 User Space

User space 包含 CA 和 libopentee.so。

CA 端的 log 由应用开发者自己编写和维护，例如，如果是 Yocto 域的应用，则可以用 printf 打印，如果是 Android 域的应用，则由 ALOG 等 Android 指定的 log 方式输出。

注：TEEC_OpenSession 和 TEEC_InvokeCommand 这两个 API 的最后一个参数 `err_origin` 可以表示错误发生在哪个阶段，强烈建议应用开发者打印出此值，例如：

```
res = TEEC_OpenSession(&ctx, &sess, &uuid,
TEEC_LOGIN_PUBLIC, NULL, NULL, &err_origin);
if (res != TEEC_SUCCESS)
    errx(1, "TEEC Opensession failed with code 0x%x origin 0x%x", res, err_origin);

res = TEEC_InvokeCommand(&sess, TA_HELLO_WORLD_CMD_INC_VALUE, &op, &err_origin);
if (res != TEEC_SUCCESS)
    errx(1, "TEEC_InvokeCommand failed with code 0x%x origin 0x%x", res, err_origin);
```

origin 的值为 1-4，其意义分别为，在 user space 层出错；Linux/atf 通讯出错；OP-TEE Kernel 层出错；TA 层出错。

```
#define TEEC_ORIGIN_API 0x00000001
#define TEEC_ORIGIN_COMMS 0x00000002
#define TEEC_ORIGIN_TEE 0x00000003
#define TEEC_ORIGIN_TRUSTED_APP 0x00000004
```

liboptee.so 与开发者编写的 CA code 链接在一起，打印方式与 CA 的打印方式相同。

4.2.2 Linux Kernel

OP-TEE Linux Kernel 中的 error log 由 pr_err 输出，打印在 dmesg 中。

4.2.3 Secure World

Secure World 包含 OP-TEE OS 和 TA 两部分。

OP-TEE OS 有定义四种 log level，分别是 FMSG/DMSG/IMSG/EMSG。

OP-TEE OS 层的 log level 默认设置为 1，即只允许 EMSG 的 log 输出。

处于安全设计，OP-TEE log level 不允许动态调整，若需要调整 log level，需要修改 OP-TEE 的配置并重新编译 OP-TEE。修改位置为：

vendor/mediatek/proprietary/trustzone/optee/4.7.0/optee_os/mk/config.mk

```
# Log levels for the TEE core. Defines which core messages are displayed
# on the secure console. Disabling core log (level set to 0) also disables
# logs from the TAs.
# 0: none
# 1: error
# 2: error + info
# 3: error + info + debug
# 4: error + info + debug + flow
CFG_TEE_CORE_LOG_LEVEL ?= 1
```

注意：

- MediaTek 是以二进制形式释放 OP-TEE 镜像，log 等级默认设置为 1，如需调整 TEE OS 的 log 等级，请联系 MediaTek。

OP-TEE OS 层新增加 Guest ID 表达当前 code 执行在哪个 VM，例如：

E/TC:1 5 0 xxxxx

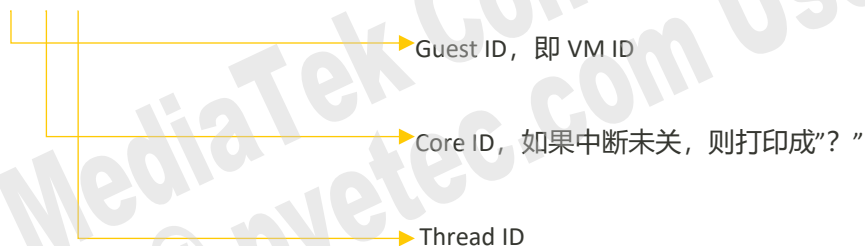


图 4-1. OP-TEE log 示例

TA 的 log 与 OP-TEE OS 设计类似，有定义四种 log level，分别是 FMSG/DMSG/IMSG/EMSG。

默认设置为 1，即只允许 EMSG 的 log 输出。

如希望调整 TA 的 log 等级，可由 TA 开发者自由完成。在 TA 的 `makefile` 中定义 `CFG_TEE_TA_LOG_LEVEL` 的值即可设定 TA 的 log level，默认为 1。

注意：

- S-EL0(TA)和 S-EL1(OPTEE)在 Secure World 的 log 会传回对应域的 Kernel 层，在 Linux Kernel 进行输出，输出等级为 `pr_info`。可以通过 `dmesg` 等方式抓取 Kernel 的 log，其中会包含 OP-TEE 在 S-EL0/1 的 LOG。

例如：

```
adb shell dmesg -w > log.txt
```

4.3 OP-TEE 常见异常与分析

开源社区有文档介绍 OP-TEE 的 abort dump 和 call stack 如何分析和处理，请参阅：

https://optee.readthedocs.io/en/latest/debug/abort_dumps.html

本章节基于此链接内容做简单描述。

当 TEE OS 或 TA 发生 abort 或 panic 时，会打印 panic 的 log

当 log 中出现 “TEE load address”和 “call stack” 时，表示异常出现在 OP-TEE OS 中。

```
E/TC:2 1 TEE load address @ 0x6cc30000
E/TC:2 1 Call stack:
E/TC:2 1 0x6ccb5fe0
```

OP-TEE OS 的 panic 信息会由 MediaTek 分析。

当异常出现在开发者编写的 TA 中时，需要有开发者自行分析错误原因。

这里我们直接拿开源社区文档中的例子做说明，当 call stack 中出现 Status of TA 时，表明异常是发生在 TA 中。

```

E/TC:0 TA panicked with code 0x0
E/TC:0 Status of TA 484d4143-2d53-4841-3120-4a6f636b6542 (0xe07ba50) (active)
E/TC:0 arch: arm load address: 0x101000 ctx-idr: 1
E/TC:0 stack: 0x100000 4096
E/TC:0 region 0: va 0x100000 pa 0xe31d000 size 0x1000 flags rw-
E/TC:0 region 1: va 0x101000 pa 0xe300000 size 0xf000 flags r-x
E/TC:0 region 2: va 0x110000 pa 0xe30f000 size 0x3000 flags r--
E/TC:0 region 3: va 0x113000 pa 0xe312000 size 0xb000 flags rw-
E/TC:0 region 4: va 0 pa 0 size 0 flags ---
E/TC:0 region 5: va 0 pa 0 size 0 flags ---
E/TC:0 region 6: va 0 pa 0 size 0 flags ---
E/TC:0 region 7: va 0 pa 0 size 0 flags ---
E/TC:0 Call stack:
E/TC:0 0x001044a8
E/TC:0 0x0010ba59
E/TC:0 0x00101093
E/TC:0 0x001013ed
E/TC:0 0x00101545
E/TC:0 0x0010441b
E/TC:0 0x00104477

```

第一种分析方式为采用开源社区提供的脚本直接做解析，推荐优先使用这种方式

```
$ cat dump.txt | ./optee_os/scripts/symbolize.py -d ./optee_examples/*/ta
```

若第一种方式解析不出问题所在，也可以尝试手动解析，解析方式为：

找到此 TA 对应的 {uuid}.dmp 文件，

如果是基于 Yocto 环境编译，dmp 文件在 build/tmp/work/xxxx 下面

如果是基于 Android 环境编译，dmp 文件在 out 下面

第一步，计算 offset，方式为 crash 是的地址，减去 TA 的 load address，例如

```
offset = 0x001044a8 - 0x101000 = 34A8
```

第二步，使用文本工具打开 {uuid}.dmp

在 {uuid}.dmp 中找到 34A8 对应的位置，即可知道大致发生 panic 的位置

5 替换 OP-TEE TA 的签名与验签密钥

出于安全性设计，OP-TEE 会在加载 TA 时校验其签名，若校验失败，则拒绝 TA 的载入。

MTK 在 release OP-TEE 时，会将 OP-TEE TA 验签 key 替换为 MTK 派生的密钥。

处于安全性考虑，MTK 建议客户替换并自行维护 TA 验签密钥。

本章将介绍 OP-TEE 4.7.0 版本中，如何替换 TA 验签密钥。

5.1 OP-TEE TA 签名验签简介

在[章节 2.4](#)中有介绍三种类型的 TA: UTA, PTA 和 Early UTA。

其中 PTA 和 Early UTA 会被打包进 tee.img，由 secure boot 验签 tee.img 时保证其安全性。

普通 UTA 则以{UUID}.ta 的文件方式保存在文件系统中，在编译{UUID}.ta 时会进行签名。

在 OPTEE 从文件系统中加载{UUID}.ta 时，需校验签名后才可载入安全内存中执行。

验签 key 使用 RSA 4096，公钥会在编译 OP-TEE OS 时链接进 OP-TEE 镜像中；而私钥存放在编译服务器的 OP-TEE TA 开发包中，在编译 TA 时用于签名 TA。

5.2 准备工作

1. 确认您的 branch 上已经存在如下 patch：

分别在 Android 和 Yocto 的 code 中 grep OPTEE_CUSTOMIZED_TA_SIGN_KEY，若无此关键字，则说明您的 codebase 还未包含相关 patch，需要先联系 MTK 释放相关 patch。

- Android branch 的 grep 目录为 vendor/mediatek/proprietary/trustzone/custom/build
- Yotco branch 的 grep 目录为 meta/meta-mediatek

2. 准备好新的密钥，其步骤为：

在 linux server 上依次执行以下 5 个命令

```
openssl genpkey -algorithm RSA -out private_key.pem -pkeyopt rsa_keygen_bits:4096
```

```
openssl rsa -pubout -in private_key.pem -out public_key.pem
```

```
openssl rsa -pubin -inform PEM -in public_key.pem -outform DER -out public_key.der
```

```
openssl asn1parse -in public_key.der -inform DER -strparse 19 -out n_value.der
```

```
dd if=n_value.der of=test_rsa_pub_n.bin bs=1 skip=9 count=256
```

执行完后，最终会得到如下 5 个文件

```
-rw-rw-r-- 1 mtk mtk 270 Jun 11 11:21 n_value.der
-rw----- 1 mtk mtk 1704 Jun 11 11:20 private_key.pem
-rw----- 1 mtk mtk 294 Jun 11 11:20 public_key.der
-rw-rw-r-- 1 mtk mtk 451 Jun 11 11:20 public_key.pem
-rw-rw-r-- 1 mtk mtk 256 Jun 11 11:21 test_rsa_pub_n.bin
```

接下来，请依据[章节 5.3](#)，将 test_rsa_pub_n.bin 放入 tee.img 中；依据[章节 5.4](#)，需要将 private_key.pem 放于 TA 签名脚本中；[章节 5.5](#) 则介绍如何在不重新编译 TA 的前提下重新签名 TA。

5.3 替换 OP-TEE 镜像中的验证密钥

MTK 以 binary 方式释放 OP-TEE 镜像，在编译时，OP-TEE 镜像会与 ATF 镜像一起打包成 tee.img。

我们需要在打包过程中，将 test_rsa_pub_n.bin 替换进 tee.img 中，从而达到替换公钥的目的。

TEE 的镜像从 Android branch 中编译出，故此处要修改的是 Android branch。

Android branch:

步骤 1: 将[章节 5.2](#) 生成的 test_rsa_pub_n.bin 放置在 code tree 中，建议路径为：

```
vendor/mediatek/proprietary/trustzone/custom/build/keys/test_rsa_pub_n.bin
```

步骤 2: 在 project config 中设定 key 的位置

示例：在 vendor/mediatek/proprietary/trustzone/custom/build/project/{PROJECT_NAME}.mk

(PROJECT_NAME 需替换成您使用的 project，例如 auto8668p1_64_vm_uos) 中添加

```
OPTTEE_CUSTOMIZED_TA_SIGN_KEY := $(TRUSTZONE_CUSTOM_BUILD_PATH)/keys/test_rsa_pub_n.bin
```

步骤 3: 编译产生新的 tee.img

注意：

- xxx.mk 文件替换为您的 project 所使用的文件
- 步骤 2 中设定的路径需要与步骤 1 中文件摆放路径一致。

可使用如下指令单独编译 tee.img 以提高编译速度

```
python3 vendor/mediatek/proprietary/scripts/releasetools/split_build_helper.py
full_{PROJECT_NAME}-userdebug --run --layers vext --target trustzone
```

编译产物为 out/target/product/{PROJECT_NAME}/tee-verified.img

PROJECT_NAME 需替换成您使用的 project，例如 auto8668p1_64_vm_uos

5.4 替换私钥

5.4.1 Yocto 侧替换私钥

步骤 1: 将[章节 5.2](#) 中生成的 `private_key.pem` 重命名为 `ta_sign_key.pem` 后放在指定位置

以 MT8668 为例，路径为

```
meta/meta-mediatek/recipes-bsp/trustzone/keys/ta_sign_key.pem
```

步骤 2: 在 `project config` 中使能 `OPTEE_CUSTOMIZED_TA_SIGN_KEY`

以 MT8668 为例，路径为

```
meta/meta-mediatek-mt8668-hyp/conf/machine/{PROJECT_NAME}.conf
```

(`PROJECT_NAME` 需替换成您使用的 `project`，例如 `auto8668p1_64_sos`)

新增 `config`:

```
OPTEE_CUSTOMIZED_TA_SIGN_KEY = "yes"
```

完成以上步骤后，再编译 TA 时就会使用新的 Key 签名 TA。

5.4.2 Android 侧替换私钥

在编译 TA 时需要用到 OP-TEE TA 开发包，其路径为：

```
vendor/mediatek/proprietary/trustzone/optee/4.7.0/optee_os_spmllib/tz_optee_4.7.0_dev_kit.tar.gz
```

签名 TA 的 key 位于 `export-ta_arm64/keys/default_ta.pem`

将[章节 5.2](#) 生成的 `private_key.pem` 重命名为 `default_ta.pem` 并替换原文件，

这样再编译 TA 时，就会使用新的 key 签名。

5.5 重新签名 TA

对于有 source code 的 TA，在编译 TA 的过程中会自动签名 TA，所以只需要按照[章节 5.4](#) 的介绍替换私钥，替换之后编译出的 TA 就会使用新的 Key 签名。

如果无法重新编译 TA，只想替换签名（例如 MTK binary release 的 TA），本章介绍如何替换签名。

签名 TA 需要使用 OP-TEE 提供的签名脚本 `sign_encrypt.py`，配合新的私钥，对 TA 重新签名，其用法如下
需要提前准备好“私钥”和“未签名的 TA”

```
export TA_SIGN_KEY= private_key.pem
export UUID=fd02c9da-306c-48c7-a49c-bbd827ae86ee (替换为您需要签名的 TA 的 UUID)

python3 ${TA_DEV_KIT_DIR}/scripts/sign_encrypt.py --key $TA_PUBLIC_KEY --uuid ${UUID} \
    --ta-version 0
    --in ${TA_OUT_PATH}/${UUID}.ta.unsigned \
    --out ${TA_OUT_PATH}/${UUID}.ta
```

注意:

- 如果没有“未签名的 TA”，则需要把“已签名的 TA”的前 328 Byte 移除，来得到“未签名 TA”。

接下来介绍如何重新签名 MTK 以 binary 形式 release 的 TA。

5.5.1 Yocto 侧重新签名 TA

MTK 在 Yocto 测以 binary release 的 TA 只有 pkcs11 TA。

受益于 Yocto 编译系统的灵活性，MTK 已经在 pkcs11 TA 的编译脚本中做了相关处理，只要按照[章节 5.4.1](#) 替换完私钥，再次编译时，就会自动重新签名 pkcs11 TA。

5.5.2 Android 侧重新签名 TA

而 Android 侧对 MTK binary release 的 TA，需要手动重新签名，并 merge 进 code tree 中。

同[章节 5.4.2](#) 介绍，需要先把 OP-TEE TA 开发包中的私钥替换为新的私钥。

MTK binary release 的 TA 位于如下路径

vendor/mediatek/proprietary/trustzone/optee/services

- unsigned TA: {uuid}.ta.unsigned
- MTK signed TA: {uuid}.ta

MTK 提供的 TA 列表和说明如下，根据需求不同，包含的 TA 数量可能不同。

表 5-1. MTK Android 测 TA 说明

UUID	类型	用途
5f902ace-5e5c-4cd8-ae54-87b88c22ddaf	Early UTA	Keyminit, 此 TA 无需重新签名
fd02c9da-306c-48c7-a49c-bbd827ae86ee	UTA	PKCS11
989850bf-4663-9dcd-394c-07a45f4633d2	UTA	KeyManager
edff8ba9-79d6-4ace-a3c8-27dcd51d21ed	UTA	Widevine
38ba0cdc-df0e-11e4-9869-233fb6ae4795	UTA	GateKeeper

如下为签名步骤：

- a. `cd vendor/mediatek/proprietary/trustzone/optee/4.7.0/optee_os_spmllib`
- b. `tar -zxvf tz_optee_4.7.0_dev_kit.tar.gz,`
- c. 更换 sign key (重命名并覆盖 `export-ta_arm64/keys/default_ta.pem`)
- d. `export PYTHON3="/./././././././prebuilts/clang/host/linux-x86/clang-r487747c/python3/bin/python3"`
- e. `export UUID="38ba0cdc-df0e-11e4-9869-233fb6ae4795"`
- f. `${PYTHON3} export-ta_arm64/scripts/sign_encrypt.py --key export-ta_arm64/keys/default_ta.pem --uuid ${UUID} --ta-version 0 --in ${UUID}.ta.unsigned --out ${UUID}.ta`

以上步骤一次签名一个 TA，若有多个 TA，需要设定不同的 uuid，循环步骤 d~e。

重签之后，新的 TA 会覆盖到 MTK binary release。

注意：

- 重签之后，会覆盖源文件，记得上传新签名的 TA。

5.6 验证方式

替换完密钥和重签 TA 后，建议执行一次 TA 的 open/close 动作来确认操作成功。

若 TA 签名不对，在 open session 时会返回错误码：0xFFFF000F(TEEC_ERROR_SECURITY)。

6 附录

6.1 参考文档

1. OP-TEE 开源社区文档: <https://optee.readthedocs.io/en/latest/index.html>
2. Arm 开发者文档关于 TrustZone 的部分:
<https://developer.arm.com/documentation/102418/0102?lang=en>

附件一 附加条款

您(或您所代表的公司或其他法律主体, 以下合称「您」)是否得取得、下载与使用本文件及其相关信息皆应以您接受本附加条款为先决要件。您对于本文件之使用、取得或下载即表示您已接受本附加条款并同意受本附加条款之拘束。若您不同意受本附加条款之拘束, 您将不得使用、取得或下载本文件并应立即删除或毁弃所有本文件之副本。

本文件含有联发科技股份有限公司及其关联公司(以下合称「联发科技」)或其授权人之机密信息及专有信息, 仅供您为本文件所描述之联发科技芯片组作内部使用而不得用于其他任何目的(包括但不限于确认或提供支持任何对联发科技、其供货商及/或其直接或间接客户所提潜在专利侵权主张的证据)。禁止未经授权使用或揭露本文件及其中所含信息。您同意赔偿联发科技因您未经授权使用或揭露本文件及其中所含信息之一部或全部导致联发科技所受之任何损失或损害。

联发科技及其授权人保有本文件的所有权及组成所有权之各项权利且未针对任何知识产权授权(无论明示或默示、透过禁反言原则或其他方式)。联发科技得随时变更本文件内容而无须另行通知。联发科技无须承担与使用或信赖本文件相关或因使用或信赖本文件致生之任何责任, 包括但不限于间接损害或附带损害赔偿責任。

本文件及联发科技所提供关于本文件之任何其他材料、信息或技术支持, 均以现况交付为准, 联发科技不负任何明示、默示、法定或其他形式之担保责任, 特别是适销性、不侵权、针对特定用途之适用性、完整性或正确性之担保责任或任何由贸易惯例或交易、履行过程所生之担保责任。对于联发科技为符合您所提规格或遵循特定标准或标准组织的要求所作之交付物, 联发科技亦不承担任何责任。

于未对前述条款造成限制之情形下, 联发科技不对其产品任何特定用途之适用性承担任何保证或担保责任, 亦不承担任何因产品、电路或软件之应用或使用致生之赔偿责任。您同意您应单独承担关于整合您的产品与联发科技产品所涉及之设计、验证与测试之所有责任, 并确保前述整合产品符合相关标准及任何安全性要求或其他要件。

本附加条款及所有与本附加条款或本文件相关之行为应以台湾法律管辖、解释与阐明, 不适用冲突法原则。